

CSE-3203: Design and Analysis of Algorithms-II
University of Dhaka — B.Sc. Final Examination 2021
Complete Worked Solutions

The exam asks students to answer any five of the seven questions. For completeness, full solutions to all seven are given below.

Question 1

(a) Double hashing insertion (6 marks)

Table size $m = 11$, $h_1(k) = k \bmod 11$ (the given $h_1(k) = k$ is reduced mod m when used as a table index), $h_2(k) = 1 + (k \bmod (m - 1)) = 1 + (k \bmod 10)$.

Probe sequence: $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod 11$, $i = 0, 1, 2, \dots$

Key k	$h_1(k)$	$h_2(k)$	Probe sequence	Placed at
10	10	1	10	10
22	0	3	0	0
31	9	2	9	9
4	4	5	4	4
15	4	6	4(occ)→10(occ)→5	5
28	6	9	6	6
17	6	8	6(occ)→3	3
88	0	9	0(occ)→9(occ)→7	7
59	4	10	4(occ)→3(occ)→2	2

Resulting hash table:

Index	0	1	2	3	4	5	6	7	8	9	10
Key	22	–	59	17	4	15	28	88	–	31	10

Slots 1 and 8 remain empty; load factor $\alpha = 9/11 \approx 0.818$.

(b) Is 4 probes an accurate average verdict bound? (4 marks)

Empirically (from part a): the number of probes used to *find* each key equals the number of probes used during its insertion:

10:1, 22:1, 31:1, 4:1, 15:3, 28:1, 17:2, 88:3, 59:3

Average successful-search probes = $\frac{1 + 1 + 1 + 1 + 3 + 1 + 2 + 3 + 3}{9} = \frac{16}{9} \approx 1.78$, well under 4.

But a “verdict” also has to cover the *not-found* case, and this is where the statement breaks down. Under the standard uniform-hashing assumption, for open addressing with load factor $\alpha = n/m$:

$$\text{Unsuccessful search: } E[\text{\#probes}] \leq \frac{1}{1 - \alpha}, \quad \text{Successful search: } E[\text{\#probes}] \leq \frac{1}{\alpha} \ln \frac{1}{1 - \alpha}.$$

Here $\alpha = 9/11 \approx 0.818$, so

$$\frac{1}{1 - \alpha} = \frac{1}{2/11} = 5.5, \quad \frac{1}{\alpha} \ln \frac{1}{1 - \alpha} = \frac{11}{9} \ln(5.5) \approx 1.222 \times 1.705 \approx 2.08.$$

Conclusion: I *disagree* with the statement as a blanket claim. Successful searches are cheap here (empirically ≈ 1.8 , theoretically ≈ 2.1 probes), safely under 4. But an *unsuccessful* search — needed to report “not found” — costs on average about 5.5 probes at this load factor, which exceeds 4. Because the table is quite full ($\alpha \approx 0.82$, close to 1), the “within 4 probes on average” guarantee cannot hold uniformly for every possible query; it only holds for keys that are actually present.

(c) Chaining: average-case successful search is $\Theta(1 + \alpha)$ (4 marks)

Setup. A hash table T with m slots holds n keys via chaining; load factor $\alpha = n/m$. Under simple uniform hashing, each key is equally likely to hash to any of the m slots, independently.

Claim. The expected number of elements examined in a successful search is $\Theta(1 + \alpha)$.

Proof. Consider searching for a key x that was the i -th key inserted (in insertion order $k_1, \dots, k_n$), and suppose the elements are appended to the end of their chain on insertion. The number of elements examined equals 1 (for x itself) plus the number of elements inserted *before* x that hash to the same slot as x .

For $j < i$, let X_{ij} be the indicator random variable that k_i and k_j collide, i.e. $X_{ij} = 1$ if $h(k_i) = h(k_j)$. By simple uniform hashing, $\Pr[X_{ij} = 1] = 1/m$, so $E[X_{ij}] = 1/m$.

The expected number of elements examined when searching for the i -th inserted key is

$$E\left[1 + \sum_{j=1}^{i-1} X_{ij}\right] = 1 + \sum_{j=1}^{i-1} E[X_{ij}] = 1 + \frac{i-1}{m}.$$

Averaging over all n keys (each equally likely to be the one searched for, in a successful search):

$$\frac{1}{n} \sum_{i=1}^n \left(1 + \frac{i-1}{m}\right) = 1 + \frac{1}{nm} \sum_{i=1}^n (i-1) = 1 + \frac{1}{nm} \cdot \frac{n(n-1)}{2} = 1 + \frac{n-1}{2m}.$$

Since $n-1 < n$ and $\alpha = n/m$,

$$1 + \frac{n-1}{2m} = 1 + \frac{\alpha}{2} - \frac{1}{2m} = \Theta(1 + \alpha).$$

Hence the average-case time for a successful search is $\Theta(1 + \alpha)$. ■

Question 2

(a) Convex hull via Graham Scan (6 marks)

Points: $(-7, 8), (-4, 6), (2, 6), (8, 6), (-10, 4), (-2, 2), (-8, -2), (3, -2), (7, -2), (-9, -5), (0, -6), (8, -7), (6, -10)$.

Step 1 — Pick the pivot. Choose the point with the smallest y -coordinate: $M = (6, -10)$.

Step 2 — Sort by polar angle around M . Computing angles of every other point relative to M gives the order (increasing angle, i.e. sweeping counterclockwise from the positive x -direction):

$L(8, -7), I(7, -2), D(8, 6), C(2, 6), H(3, -2), B(-4, 6), F(-2, 2), A(-7, 8), E(-10, 4), K(0, -6), G(-8, -2), J$

(Note $I(7, -2)$ and $D(8, 6)$ are collinear with M along the same ray, since $M \rightarrow I = (1, 8)$ and $M \rightarrow D = (2, 16) = 2 \times (1, 8)$.)

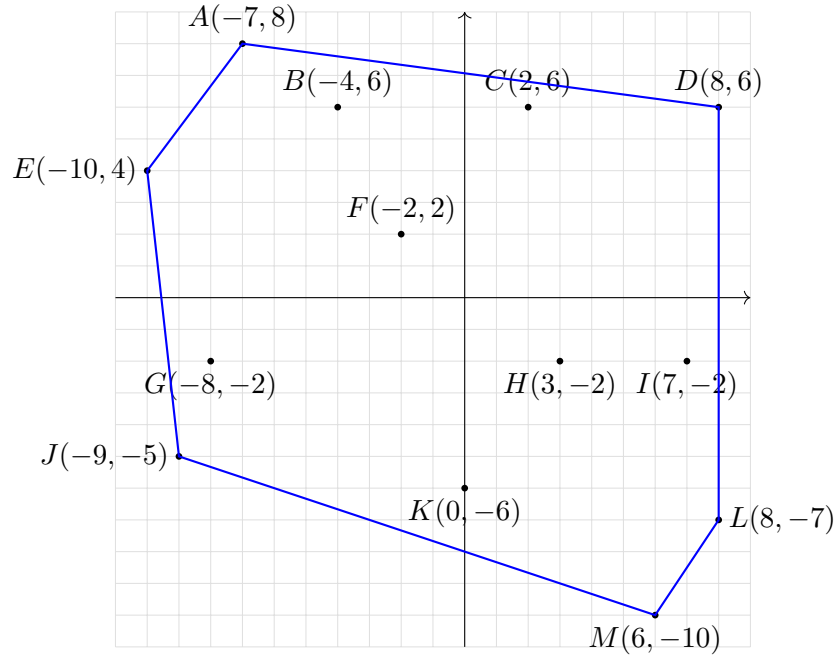
Step 3 — Scan, keeping only left turns. Push M , then L . For each next point, pop the stack while the last three points make a non-left (clockwise or straight) turn, then push the new point.

- Push I : M, L, I turns left — keep.
- Try D : turn $L \rightarrow I \rightarrow D$ is a *right* turn — pop I . Turn $M \rightarrow L \rightarrow D$ is left — push D . Stack: M, L, D .
- Push C : $L \rightarrow D \rightarrow C$ left — keep. Stack: M, L, D, C .
- Try H : $D \rightarrow C \rightarrow H$ left — push. Stack: M, L, D, C, H .
- Try B : $C \rightarrow H \rightarrow B$ is a right turn — pop H . Then $D \rightarrow C \rightarrow B$ is *straight* (collinear, all on $y = 6$) — pop C . Then $L \rightarrow D \rightarrow B$ is left — push B . Stack: M, L, D, B .
- Try F : $D \rightarrow B \rightarrow F$ left — push. Stack: M, L, D, B, F .
- Try A : $B \rightarrow F \rightarrow A$ right — pop F . $D \rightarrow B \rightarrow A$ right — pop B . $L \rightarrow D \rightarrow A$ left — push A . Stack: M, L, D, A .
- Push E : $D \rightarrow A \rightarrow E$ left — keep. Stack: M, L, D, A, E .
- Try K : $A \rightarrow E \rightarrow K$ left — push, then try G : $E \rightarrow K \rightarrow G$ right — pop K . $A \rightarrow E \rightarrow G$ left — push G .
- Try J : $E \rightarrow G \rightarrow J$ right — pop G . $A \rightarrow E \rightarrow J$ left — push J . Stack: M, L, D, A, E, J .

Result — the hull (in CCW order):

$$(6, -10) \rightarrow (8, -7) \rightarrow (8, 6) \rightarrow (-7, 8) \rightarrow (-10, 4) \rightarrow (-9, -5) \rightarrow (6, -10).$$

All remaining points (I, C, H, B, F, K, G) lie strictly inside this hexagon.



(b) Kirkpatrick–Seidel runs in $O(n \lg h)$ (4 marks)

The Kirkpatrick–Seidel (“ultimate”) algorithm computes the upper hull (the lower hull is symmetric) using a *marriage-before-conquest* / *prune-and-search* strategy:

1. **Bridge-finding in linear time.** Given a set of n points and a vertical line $x = x_0$ splitting them roughly in half, the algorithm finds the *bridge edge* of the upper hull that crosses x_0 in $O(n)$ time. This is done using a linear-time weighted median algorithm together with a pairing/slope-elimination technique: points are paired up, their connecting slopes are computed, the median slope is found in $O(n)$, and depending on which side of the bridge the median-slope line lies, at least a constant fraction ($1/4$) of the points are discarded from further consideration — all in $O(n)$ time (standard “prune and search”).
2. **Recursive structure.** Once the bridge for the current point set is found, points strictly below the bridge play no role in the upper hull and are discarded. The two endpoints of the bridge split the remaining points into a left subproblem (points left of the left bridge endpoint) and a right subproblem (points right of the right bridge endpoint); each subproblem’s hull is found recursively.
3. **Output-sensitivity.** Crucially, unlike a straightforward divide and conquer that always recurses on $n/2$ points each side (giving $O(n \log n)$), here the two recursive subproblems’ sizes are *not necessarily equal*; instead, what matters is that each hull vertex found corresponds to one “successful” branch of the recursion.

Recurrence and analysis. Let h be the number of hull vertices actually output. Build a *recursion tree* where each internal node does $O(n_v)$ work (n_v = number of points at that node) to find one bridge point, and each node has at most 2 children. Because every internal node of this recursion tree contributes exactly one hull edge/vertex to the final answer, and the tree has at most h leaves (one per hull vertex found), the tree has depth $O(\lg h)$: a binary tree that ultimately identifies h items via a divide-and-discard process of the above kind has depth $O(\lg h)$ (each level at least halves — in the worst-case branch — the remaining “hull-vertex budget”).

At each level of this recursion tree, the total work summed over all nodes at that level is $O(n)$,

because:

- Each node does $O(n_v)$ work for its own point set of size n_v ;
- The point sets at a given level are disjoint subsets of the original n points (each point charged to at most one node per level, since discarded points never recur), so $\sum n_v \leq n$ at every level.

Since there are $O(\lg h)$ levels, each costing $O(n)$ total work,

$$T(n, h) = O(n \lg h).$$

This matches the information-theoretic lower bound $\Omega(n \lg h)$ for computing the convex hull of n points with h hull vertices, so Kirkpatrick–Seidel is asymptotically optimal. ■

(c) $O(n^2 \lg n)$ collinearity test (4 marks)

Goal: decide if any 3 of n points are collinear.

Algorithm.

1. For each point p_i ($i = 1, \dots, n$):
 - (a) Compute the slope (or, to avoid division, the angle / reduced-fraction direction) from p_i to every other point p_j — $O(n)$ time, $n - 1$ slopes.
 - (b) Sort these $n - 1$ slopes — $O(n \lg n)$ time.
 - (c) Scan the sorted slopes once: if any two *adjacent* slopes are equal, then p_i together with those two corresponding points are collinear — report YES. This scan is $O(n)$.
2. If no pivot p_i ever finds two equal slopes, report NO 3 points are collinear.

Correctness. Three points p_i, p_j, p_k are collinear iff the slope of segment $p_i p_j$ equals the slope of segment $p_i p_k$ (treating p_i as pivot). So collinearity of *any* triple is detected when we pivot on any one of its three points (in particular when we pivot on p_i , the smallest-indexed point of the triple) — comparing slopes from that pivot to the other two suffices, and equal slopes must be adjacent after sorting.

Running time. Outer loop runs n times; each iteration costs $O(n \lg n)$ (dominated by sorting). Total: $O(n \cdot n \lg n) = O(n^2 \lg n)$. ■

Question 3

(a) KMP prefix function for “abbababba” (6 marks)

Pattern (0-indexed): $P = a b b a b a b b a$.

Using the standard prefix-function computation ($k \leftarrow \pi[k - 1]$ on mismatch):

i	0	1	2	3	4	5	6	7	8
$P[i]$	a	b	b	a	b	a	b	b	a
$\pi[i]$	0	0	0	1	2	1	2	3	4

Step-by-step trace:

- $i = 0$: $\pi[0] = 0$ (by definition).
- $i = 1$: $k = 0$; $P[0]=a \neq P[1]=b \Rightarrow \pi[1] = 0$.
- $i = 2$: $k = 0$; $P[0]=a \neq P[2]=b \Rightarrow \pi[2] = 0$.
- $i = 3$: $k = 0$; $P[0]=a = P[3]=a \Rightarrow k = 1$, $\pi[3] = 1$.
- $i = 4$: $k = 1$; $P[1]=b = P[4]=b \Rightarrow k = 2$, $\pi[4] = 2$.
- $i = 5$: $k = 2$; $P[2]=b \neq P[5]=a$; fall back $k = \pi[1] = 0$; $P[0]=a = P[5]=a \Rightarrow k = 1$, $\pi[5] = 1$.

- $i = 6$: $k = 1$; $P[1]=b = P[6]=b \Rightarrow k = 2$, $\pi[6] = 2$.
- $i = 7$: $k = 2$; $P[2]=b = P[7]=b \Rightarrow k = 3$, $\pi[7] = 3$.
- $i = 8$: $k = 3$; $P[3]=a = P[8]=a \Rightarrow k = 4$, $\pi[8] = 4$.

(Check: $\pi[8] = 4$ means the longest proper prefix-suffix of “abbababba” is “abba”, which is correct.)

(b) Rabin–Karp mod 11 (4 marks)

Text: 3141592653589793 (length 16). Pattern: “26”, numeric value 26, so

$$p = 26 \bmod 11 = 4.$$

Compute $t_i = (\text{2-digit substring at } i) \bmod 11$ for $i = 0, \dots, 14$:

i	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
sub	31	14	41	15	59	92	26	65	53	35	58	89	97	79	93
t_i	9	3	8	4	4	4	4	10	9	2	3	1	9	2	5

$t_i = p = 4$ at $i = 3, 4, 5, 6$ (substrings “15”, “59”, “92”, “26”). Rabin–Karp performs a character-by-character verification *only when the hashes match*, so

character-by-character checking is triggered 4 times

(at shifts $i = 3, 4, 5, 6$) — three of these (“15”, “59”, “92”) are *spurious hits* that fail verification, and the fourth ($i = 6$, “26”) is the genuine match.

(c) Minimal automaton accepting either pattern (4 marks)

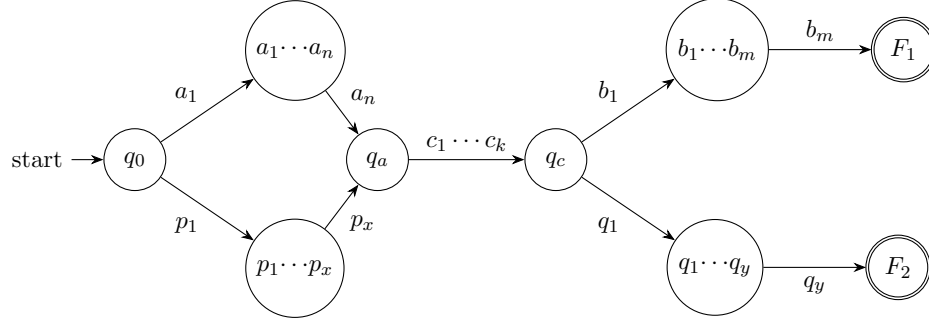
We need a machine that accepts exactly the strings

$$P_1 = a_1 a_2 \dots a_n c_1 c_2 \dots c_k b_1 b_2 \dots b_m \quad \text{or} \quad P_2 = p_1 p_2 \dots p_x c_1 c_2 \dots c_k q_1 q_2 \dots q_y,$$

which *share the identical middle block* $c_1 c_2 \dots c_k$. A naive automaton (e.g., built as two separate chains, or as a trie) would use $(n + k + m)$ states for P_1 and $(x + k + y)$ states for P_2 — i.e. $n + x + 2k + m + y$ states if the c -block is duplicated.

Key idea (state minimization): since the substring $c_1 \dots c_k$ is common to both patterns and occurs in the *same relative position* (immediately after the first block, immediately before the last block), the automaton can *share one copy* of the k states for the c -block, rather than build it twice. The construction:

- A start state q_0 .
- Two disjoint chains of new states reading $a_1, \dots, a_n$ and $p_1, \dots, p_x$ respectively, both starting at q_0 and both converging into a single common state q_a (the state reached right after consuming either full a -prefix or full p -prefix).
- From q_a , a single shared chain of k states reading $c_1, c_2, \dots, c_k$, ending at state q_c .
- From q_c , two disjoint chains reading $b_1, \dots, b_m$ (ending in accepting state F_1) and $q_1, \dots, q_y$ (ending in accepting state F_2).



State count. This uses $1 + n + x + k + m + y$ states (each chain box above expands into its own sequence of states), instead of $n + x + 2k + m + y + 2$ for two independently-built chains — saving the duplicate copy of the k shared states. This is minimal because: (i) the a -prefix and p -prefix must be kept as separate chains since in general they differ symbol-by-symbol (no sharing possible unless prefixes literally coincide), (ii) the c -block, being identical in both patterns and in the same position, can and should be merged into one chain, and (iii) the suffixes b and q again must diverge into separate chains (again, since they differ). Any further merging would conflate strings that must be distinguished.

Question 4

(a) Permutations with duplicate characters (6+4 marks)

(i) $O(n^2 \cdot n!)$ algorithm (allow duplicates in generation, filter after):

```
DISTINCT-PERMUTATIONS(S[1..n]):
```

```
    Results = empty list                // will hold only distinct perms
    GENERATE-ALL(S, 1)
    return Results
```

```
GENERATE-ALL(S, k):
```

```
// standard permutation generator
```

```
    if k == n:
        if S is not already present in Results:    // O(n) per stored perm,
            append COPY(S) to Results              // O(n) comparisons x |Results|
        return
    for i = k to n:
        swap S[k], S[i]
        GENERATE-ALL(S, k+1)
        swap S[k], S[i]    // backtrack
```

Complexity. Ignoring duplicates, the swap-based generator produces all $n!$ arrangements (with repeats when characters repeat), each taking $O(n)$ to materialize/copy. For each of the $n!$ generated arrangements we do a duplicate check against the (up to $n!$) strings already stored, each comparison costing $O(n)$: in the worst case this membership test costs $O(n \cdot n!)$ per insertion, so across all $n!$ generated strings the check dominates at $O(n \cdot n! \cdot n) = O(n^2 \cdot n!)$ overall (the generation itself is only $O(n \cdot n!)$, but the duplicate-filtering with a linear scan against a growing list of size up to $n!$ is what forces the $n^2 \cdot n!$ bound). Hence total time is $O(n^2 \cdot n!)$.

(ii) $O(n \cdot n!)$ algorithm for a string with *no* duplicate characters:

```

ALL-PERMUTATIONS(S[1..n]):
    Results = empty list
    PERMUTE(S, 1)
    return Results

PERMUTE(S, k):
    if k == n:
        append COPY(S) to Results          // O(n) to copy
        return
    for i = k to n:
        swap S[k], S[i]
        PERMUTE(S, k+1)
        swap S[k], S[i]

```

Why $O(n \cdot n!)$. With all characters distinct, every leaf of the recursion tree corresponds to exactly one of the $n!$ permutations (no duplicate filtering is needed — the swap-based generation with all-distinct symbols never re-generates the same arrangement twice). The recursion tree has $n!$ leaves; the work done *along the root-to-leaf path* that materializes a permutation is $O(n)$ (the copy step), and the total number of internal recursive calls is also $\Theta(n!)$ (standard result: $\sum_{k=0}^{n-1} \frac{n!}{k!} \cdot 1 = \Theta(n!)$ calls, each doing $O(1)$ work aside from the leaf copy). Hence total time is $O(n!) \times O(n) = O(n \cdot n!)$.

(b) Backtracking for 3-Queens (4 marks)

Place queens column by column, one per row, backtracking on conflicts (row 1 = column of queen in row 1, etc.), $N = 3$:

- **Row 1:** try column 1. Queen at (1,1).
 - **Row 2:** column 1 attacked (same column) — skip. Column 2 attacked (diagonal from (1,1)) — skip. Column 3: safe. Queen at (2,3).
 - * **Row 3:** column 1 attacked (diagonal from (2,3): $|3 - 1| = 2 \neq 1$... check: diag distance from (1,1) is $|1 - 1| = 0$ col diff vs row diff 2, not diagonal; but column 1 already used by row1 $\rightarrow$ skip). Column 2 attacked (diagonal from (2,3), $|2 - 3| = 1$, row diff 1 $\rightarrow$ attacked) — skip. Column 3 attacked (same column as row 2) — skip. **No valid column** $\rightarrow$ backtrack.
 - Row 2 has no more columns to try $\rightarrow$ backtrack to Row 1.
- **Row 1:** try column 2. Queen at (1,2).
 - **Row 2:** column 1 attacked (diagonal, row/col diff both 1) — skip. Column 2 attacked (same column) — skip. Column 3 attacked (diagonal) — skip. **No valid column** $\rightarrow$ backtrack.
- **Row 1:** try column 3 (symmetric to column 1 case) — again every branch in Row 2/Row 3 dead-ends.

Conclusion. All branches of the backtracking search tree dead-end: **the 3-Queens problem has no solution.** (This matches the well-known fact that N -Queens is unsolvable exactly for $N = 2$ and $N = 3$.)

Question 5

(a) Competitive ratio of second-chance vs. OPT (4 marks)

Let k = number of page frames.

Second-chance is a marking algorithm. A page-replacement algorithm is a *marking algorithm* if it never evicts a "marked" page within a phase (a maximal run of requests containing exactly k distinct pages), where a page is marked the moment it is requested/accessed. Second-chance fits this: it only evicts a page whose second-chance bit is unset (i.e., not recently "given a chance" / accessed), analogous to marking.

Standard theorem: every marking algorithm is k -competitive: in any phase (containing at most k distinct page faults for the marking algorithm, since at most k marked pages can be resident), OPT faults at least once per phase (since a phase by definition requests k distinct pages, more than fit in cache together with whatever was resident, so a fault is forced), while the marking algorithm faults at most k times in that phase. Hence

$$(\text{cost of algorithm}) \leq k \cdot (\text{cost of OPT}) \Rightarrow \text{competitive ratio} \leq k.$$

Matching lower bound. No deterministic online paging algorithm can do better than k -competitive (an adversary can always request the one page not currently in the algorithm's cache, forcing a fault every step, while OPT — with foresight — faults only once every k requests). Second-chance is subject to the same adversarial construction.

Conclusion: the worst-case competitive ratio of second-chance is exactly

$$\boxed{k}$$

matching the lower bound for *any* deterministic paging algorithm — second-chance is optimally competitive (in the worst-case ratio sense), even though it can be far from optimal on specific inputs.

(b) Parallel Fibonacci (4 marks)

P-FIB(n):

```
if n <= 1:
    return n
x = spawn P-FIB(n-1)    // fork a new (child) thread to compute F(n-1)
y = P-FIB(n-2)          // parent thread computes F(n-2) itself, in parallel
sync                    // parent waits for the spawned child to finish
return x + y
```

Thread interaction. The call `spawn` creates a new, logically-parallel child thread that begins executing P-FIB($n-1$) while the *parent* thread, without waiting, immediately proceeds to compute P-FIB($n-2$) itself. Both threads may run simultaneously on different processors. The `sync` statement is a barrier: the parent thread cannot proceed past it (cannot execute `return x+y`) until *all* of its previously spawned children (here, just the one computing x) have completed and written back their result. Recursively, each call spawns a new subtree of threads, so the overall computation is a binary tree of $\Theta(F(n))$ threads.

Complexity. Work $T_1(n) = \Theta(\phi^n)$ (same as serial Fibonacci recursion); span (critical-path length) $T_\infty(n) = \Theta(n)$ (the longest chain of dependent `spawns`, one per decrement of n); parallelism $= T_1/T_\infty = \Theta(\phi^n/n)$.

(c) Greedy scheduler: $T_P \leq 2T_P^*$ (6 marks)

Let T_1 = total work, T_∞ = span (critical path length) of a multithreaded computation, and T_P = time taken by a *greedy scheduler* on P processors (a greedy scheduler never leaves a processor idle if there is a ready-to-execute strand). Let T_P^* denote the time of an *optimal* scheduler on P processors.

Lower bounds on T_P^* . Any scheduler, optimal or not, satisfies

$$T_P^* \geq T_1/P \quad (\text{work bound: } P \text{ processors can do at most } P \cdot T_P^* \text{ total work}),$$

$$T_P^* \geq T_\infty \quad (\text{span bound: the critical path must still be executed in order, regardless of } P).$$

Hence $T_P^* \geq \max(T_1/P, T_\infty)$.

Upper bound on greedy T_P (CLRS Theorem). Classify each of the T_P time steps as either *complete* (all P processors are busy) or *incomplete* (at least one processor is idle).

- **Complete steps.** Each complete step performs P units of work. If there were more than T_1/P complete steps, total work done would exceed T_1 , a contradiction. So the number of complete steps is at most T_1/P .
- **Incomplete steps.** At an incomplete step, since the scheduler is greedy, *every* strand that is ready to run *is* run (nothing is held back), which means every strand whose predecessors have completed is currently executing. Consider the subgraph of the DAG not yet executed; an incomplete step must execute every strand with in-degree 0 in that remaining DAG, so it strictly reduces the length of the longest remaining path by exactly 1. Since the span is initially T_∞ , there can be at most T_∞ incomplete steps.

Adding these,

$$T_P \leq \frac{T_1}{P} + T_\infty.$$

Combine. Using $T_P^* \geq \max(T_1/P, T_\infty) \geq \frac{1}{2}(T_1/P + T_\infty)$ (since the max of two nonnegative numbers is at least their average),

$$T_P \leq \frac{T_1}{P} + T_\infty \leq 2 \max\left(\frac{T_1}{P}, T_\infty\right) \leq 2T_P^*.$$

Hence any greedy scheduler is within a factor of 2 of optimal on P processors. ■

Question 6

(a) Optimal solution from decision-problem oracle? (2 marks)

Yes, for essentially all natural optimization problems whose decision version is "self-reducible." The typical technique is:

1. **Find the optimal value** via binary search on the decision predicate: repeatedly ask "does a solution with value $\leq k$ (or $\geq k$) exist?" for various k in the (polynomially-bounded, or at worst exponential-but-log-searchable) range of possible objective values. Each query is one call to the decision oracle; $O(\log(\text{range}))$ queries pin down the optimal value k^* .
2. **Reconstruct an actual optimal solution** using self-reduction: fix parts of the instance one piece at a time (e.g., for TSP, tentatively force/forbid a particular edge) and re-query the decision oracle to check whether the optimal value k^* is still achievable; keep the choice if yes, discard it if no. Repeating for every component of the solution reconstructs a full optimal solution using only polynomially many decision-oracle calls.

So: yes, an efficient decision procedure can be turned into an efficient optimization procedure, at the cost of a polynomial-factor overhead in oracle calls.

(b) $Y \leq_p X$ (4 marks)

By definition, $Y \leq_p X$ means there is a polynomial-time computable function f such that $y \in Y \iff f(y) \in X$.

(i) If $X \in \mathbf{P}$, then $Y \in \mathbf{P}$. Given input y , compute $f(y)$ in polynomial time (by definition of $\leq_p$), then run the poly-time decision algorithm for X on $f(y)$. The composition of two polynomial-time procedures is polynomial-time, and $y \in Y \iff f(y) \in X$, so this correctly decides Y in polynomial time.

(ii) If $Y \notin \mathbf{P}$, then $X \notin \mathbf{P}$. This is the logical contrapositive of (i): if X were in $\mathbf{P}$, then by part (i) Y would be in $\mathbf{P}$ too — contradiction. So $X \notin \mathbf{P}$.

(Intuition: $Y \leq_p X$ means Y is "no harder than" X ; so if X is easy, Y must be easy too; equivalently if Y is hard, X must be at least as hard.)

(c) TSP is NP-complete (4 marks)

Let $\text{TSP-DECISION} = \{(G, w, k) : G \text{ has a Hamiltonian cycle of total weight } \leq k\}$.

TSP $\in \mathbf{NP}$. Given a candidate tour (a permutation of the vertices), we can, in polynomial time, verify: (1) it visits every vertex exactly once and returns to the start (Hamiltonian cycle), and (2) its total edge weight is $\leq k$ — a simple $O(n)$ summation. So a certificate can be checked in polynomial time.

TSP is NP-hard, via reduction from the Hamiltonian-Cycle problem (known NP-complete). Given a graph $G = (V, E)$ (instance of Ham-Cycle), construct a complete weighted graph $G' = (V, E')$ with

$$w(u, v) = \begin{cases} 1 & (u, v) \in E \\ 2 & (u, v) \notin E \end{cases}$$

and set $k = |V|$. This transformation is clearly polynomial-time.

Correctness: If G has a Hamiltonian cycle, the corresponding cycle in G' uses only weight-1 edges, total weight $= |V| = k$, so $(G', w, k) \in \text{TSP-DECISION}$. Conversely, if G' has a tour of weight $\leq |V|$, since every edge weighs at least 1 and there are exactly $|V|$ edges in any tour, *every* edge used must have weight exactly 1 — meaning every edge used is in E — so the tour is a genuine Hamiltonian cycle in G .

Hence $\text{Ham-Cycle} \leq_p \text{TSP-DECISION}$, and since Ham-Cycle is NP-complete, TSP-DECISION is NP-hard. Combined with $\text{TSP} \in \mathbf{NP}$, TSP is NP-complete. ■

(d) 3-CNF-SAT $\leq_p$ Clique (4 marks)

Let $\phi = C_1 \wedge C_2 \wedge \dots \wedge C_k$ be a 3-CNF formula, each clause $C_i = (\ell_{i1} \vee \ell_{i2} \vee \ell_{i3})$.

Construction. Build graph $G = (V, E)$ with one vertex $v_{i,j}$ for each literal occurrence ($i = 1, \dots, k$ clauses, $j = 1, 2, 3$ literal position), so $|V| = 3k$. Add edge $(v_{i,j}, v_{i',j'})$ iff:

- $i \neq i'$ (the two literals are from *different* clauses), **and**
- $\ell_{i,j}$ and $\ell_{i',j'}$ are *not* negations of each other (i.e., they are consistent — can be simultaneously true).

Set $g = k$ (target clique size). This construction is clearly computable in polynomial ($O(k^2)$) time.

Correctness.

- ($\Rightarrow$) If ϕ is satisfiable, fix a satisfying assignment; each clause has at least one literal set true — pick one such "true" literal vertex per clause. These k vertices come from k distinct clauses (so pairwise the "different clause" condition holds), and no two can be negations of each other (since both are true under the same consistent assignment). So all $\binom{k}{2}$ pairs are connected — a k -clique.
- ($\Leftarrow$) If G has a clique of size k , since edges only ever join vertices from different clauses, and there are only k clauses total, the clique must contain exactly one vertex per clause. Set every literal corresponding to a clique vertex to TRUE (this is consistent — no two clique vertices are negations of each other, by the edge condition). This satisfies every clause (each contributes its chosen true literal) and hence satisfies ϕ .

So ϕ is satisfiable iff G has a clique of size k ; the reduction is correct and polynomial, showing $3\text{-CNF-SAT} \leq_p \text{Clique}$. ■

Question 7

(a) RSA is trivial to crack knowing $\phi(n)$ (4 marks)

In RSA, the public key is (n, e) and the private key is d satisfying

$$e \cdot d \equiv 1 \pmod{\phi(n)}.$$

If an attacker learns $\phi(n)$ (in addition to the public n, e), they can directly compute $d = e^{-1} \bmod \phi(n)$ using the EXTENDED-EUCLID algorithm (which runs in time polynomial in $\log n$, i.e. essentially instantly) — since $\gcd(e, \phi(n)) = 1$ is guaranteed by RSA key generation, the modular inverse exists and is efficiently computable. Once d is known, the attacker holds the complete private key and can decrypt any ciphertext / forge any signature exactly as the legitimate key holder could. This is exactly as damaging as knowing the prime factorization of n , because knowing the factorization $n = pq$ immediately gives $\phi(n) = (p-1)(q-1)$ (and conversely, knowing n and $\phi(n)$ lets one recover p, q by solving the quadratic $x^2 - (n - \phi(n) + 1)x + n = 0$, whose roots are p and q). So knowing $\phi(n)$ is computationally equivalent to knowing the factorization — either one trivially breaks RSA.

(b) Solvability of $ax \equiv b \pmod{n}$ (6 marks)

Let $d = \gcd(a, n)$.

Solvability criterion. The congruence $ax \equiv b \pmod{n}$ has a solution **iff** $d \mid b$.

Why: $ax \equiv b \pmod{n}$ has a solution iff $b \in \{ax \bmod n : x \in \mathbb{Z}\}$. The set $\{ax \bmod n\}$ is exactly the set of multiples of $d = \gcd(a, n)$ modulo n (this follows because $ax + ny$ ranges exactly over multiples of $d = \gcd(a, n)$ as x, y range over integers, by Bézout's identity). So a solution exists iff $d \mid b$.

Computing a solution (if it exists).

1. Run EXTENDED-EUCLID(a, n) to obtain $d = \gcd(a, n)$ and integers x_0, y_0 with

$$ax_0 + ny_0 = d.$$

2. Check $d \mid b$. If not, no solution exists — stop.
3. Otherwise, multiply through by b/d :

$$a\left(x_0 \cdot \frac{b}{d}\right) + n\left(y_0 \cdot \frac{b}{d}\right) = b,$$

so $x^* = \left(x_0 \cdot \frac{b}{d}\right) \bmod n$ is one particular solution.

4. **All** solutions modulo n are then

$$x_i = \left(x^* + i \cdot \frac{n}{d}\right) \bmod n, \quad i = 0, 1, \dots, d-1,$$

giving exactly d distinct solutions modulo n .

(c) What does `Extended-Euclid`(F_k, F_{k+1}) return? (4 marks)

Claim. `EXTENDED-EUCLID`(F_{k+1}, F_k) returns $d = 1$ together with Bézout coefficients related to Fibonacci numbers by

$$F_{k+1} \cdot ((-1)^{k+1} F_{k-1}) + F_k \cdot ((-1)^k F_{k+1}) = 1,$$

more simply stated: it returns $d = \gcd(F_{k+1}, F_k) = 1$, with coefficients (up to sign, depending on exact recursion convention) $x = (-1)^{k+1} F_{k-1}$ and $y = (-1)^k F_k$ (indices adjusted so $F_1 = F_2 = 1$), satisfying $F_{k+1}x + F_k y = 1$.

Proof that $\gcd(F_{k+1}, F_k) = 1$. By induction using the Euclidean algorithm's own recursion: $F_{k+1} = 1 \cdot F_k + F_{k-1}$ (since $F_{k+1} = F_k + F_{k-1}$), so $\gcd(F_{k+1}, F_k) = \gcd(F_k, F_{k-1})$. This repeats all the way down to $\gcd(F_2, F_1) = \gcd(1, 1) = 1$. Hence $\gcd(F_{k+1}, F_k) = 1$ for all k .

This is also the worst case for Euclid's algorithm: because at every step the quotient is exactly 1 (since $F_{k+1} = 1 \cdot F_k + F_{k-1}$ with $F_{k-1} < F_k$), consecutive Fibonacci numbers force the *maximum possible number of recursive calls* of Euclid's algorithm for inputs of that size — this is precisely Lamé's theorem: Euclid's algorithm on inputs of size n takes $O(\log n)$ steps in the worst case, and that worst case is realized exactly by consecutive Fibonacci numbers.

The Bézout coefficients follow the same recursive pattern as the Fibonacci numbers themselves (with alternating sign), which is the standard textbook exercise result (CLRS 31.2-7): `EXTENDED-EUCLID`(F_k, F_{k-1}) returns $d = 1$ and coefficients that are themselves (signed) Fibonacci numbers. ■